

PIXOUS TECHNOLOGIES — ESTIMATION & INVOICING API

Test Suite Documentation

Overview — End-to-End, Feature & QA Tooling Coverage

Generated: August 14, 2026

Source: TESTING.md

Test framework: Pytest + FastAPI TestClient

Contents

1. Framework & conventions

2. How to run

3. E2E and feature test coverage (this document's scope)

tests/test_e2e_job_pipeline.py

tests/test_e2e_estimation_invoice_lifecycle.py

tests/test_features.py (pre-existing)

4. Notable design decisions

5. QA Postman collection (manual/exploratory testing)

6. Known gaps / not covered by any of the three test documents

This is the top-level index for the automated test suite under `tests/`. It covers the **end-to-end/feature tests** directly and links out to the two dedicated documents for the other test categories:

- [UNIT_TESTING.pdf](#) — pure/deterministic module-level tests (`test_unit_*.py`).
- [SECURITY_TESTING.pdf](#) — authentication, authorization, injection, and file/network-boundary tests (`test_security_*.py`), plus the findings this project's audits fixed and why.

1. Framework & conventions

- **Framework:** Pytest, with FastAPI's `TestClient` (Starlette/httpx) driving the real ASGI app end-to-end.
- **Pattern:** Arrange–Act–Assert. Test names state the condition and the expected behavior (`test_<subject>_<condition>_<expected_outcome>`).
- **Isolation:** `tests/conftest.py` points `DATABASE_URL` at a throwaway temp SQLite file *before* any project module is imported, and the `fresh_database` fixture drops/recreates every table before each test. No test touches `pixous.db` / production data.
- **Mocking policy:** Only external network calls (LLM providers via the LangGraph pipeline, `requests.get` in `download_url`) and non-deterministic time (`time.time()` in the rate limiter) are mocked. Everything else — the FastAPI app, the real SQLite DB, Pydantic validators, `bcrypt` , `invoice_builder` , `pdf_builder` , `organization.py` — runs for real. The system under test is never mocked.

2. How to run

```
# Full suite (unit + security + E2E + feature)
python -m pytest tests/ -q

# Just this document's scope (E2E + feature)
python -m pytest tests/test_e2e_*.py tests/test_features.py -q

# One file
python -m pytest tests/test_e2e_job_pipeline.py -q

# With coverage
python -m pytest tests/ --cov=. --cov-report=term-missing
```

Current status (full suite): **132 passed**, 0 failed (last run: 2026-08-13). Breakdown: 42 unit ([UNIT_TESTING.pdf](#)) + 72 security ([SECURITY_TESTING.pdf](#)) + 18 E2E/feature (this document).

No `.env` secrets or API keys are required to run the suite — the pipeline's LLM calls are mocked in the E2E job test, and `conftest.py` isolates the DB. `JWT_SECRET` must still be set in the environment (loaded via `config.py`); if you have a working local `.env` this is already satisfied.

3. E2E and feature test coverage (this document's scope)

`tests/test_e2e_job_pipeline.py`

Full AI-pipeline job lifecycle over HTTP. `agents.graph.build_pipeline()` drives real calls to Mistral/Groq/Gemini — the one thing this file mocks (see §5) — everything downstream of it runs for real.

Test	Scenario
<code>test_full_job_lifecycle_from_text_submission_to_document_retrieval</code>	Create job (text input) → blocking <code>/wait</code> → poll <code>/api/jobs/{id}</code> → fetch quotation/BRD documents → unknown doc type → shows up correctly in <code>/api/documents</code> , <code>/api/documents/{base_name}/data</code> , and <code>/api/clients</code> , all backed by real DB rows <code>_run_job</code> wrote
<code>test_job_with_no_input_is_rejected_before_starting_a_pipeline_run</code>	POST <code>/api/jobs</code> with no file/url/text → 400, no pipeline ever starts
<code>test_failed_pipeline_run_surfaces_error_and_never_appears_complete</code>	A pipeline stage that reports <code>..._failed</code> surfaces the real error message; the job never appears <code>complete</code> ; its documents 404
<code>test_fetching_document_for_incomplete_job_returns_404</code>	A job still <code>running</code> → document fetch is 404, not a stale/partial result
<code>test_get_job_for_unknown_id_returns_404</code>	Unknown job id → 404
<code>test_job_endpoints_require_authentication</code>	No auth header on job creation or polling → 401

`tests/test_e2e_estimation_invoice_lifecycle.py`

Manual (non-AI) estimation path, run against the real SQLite test DB, real `invoice_builder` / `pdf_builder` — nothing mocked (no network call exists in this path).

Test	Scenario
<code>test_full_manual_estimation_to_paid_invoice_lifecycle</code>	Create manual estimation → fetch data → generate invoice → fetch invoice → mark Paid (Finance role) → download invoice PDF (<code>%PDF</code> magic bytes) → shows up in <code>/api/documents</code> → aggregated correctly in <code>/api/analytics</code>
<code>test_patch_estimation_applies_optimistic_lock_and_writes_audit_log</code>	A correct <code>version</code> patch succeeds, bumps <code>version</code> , and writes an <code>AuditLog</code> row; a stale <code>version</code> patch afterward → 409
<code>test_delete_estimation_soft_deletes_and_hides_from_listing</code>	Delete → <code>is_deleted</code> / <code>deleted_at</code> set, row hidden from <code>/api/documents</code> ; deleting again → 404
<code>test_generating_invoice_for_unknown_estimation_returns_404</code>	Unknown <code>base_name</code> → 404

Test	Scenario
<code>test_invoice_status_rejects_value_outside_allowed_set</code>	A status string outside the fixed <code>INVOICE_STATUSES</code> set → 400

`tests/test_features.py` (pre-existing)

Test	Scenario
<code>test_delete_branding_asset_clears_path</code>	Deleting a branding asset clears its DB path reference and removes the file
<code>test_delete_branding_asset_requires_admin</code>	Non-Admin role → 403
<code>test_delete_branding_asset_rejects_unknown_slot</code>	An unrecognized slot name → 400
<code>test_cancel_job_marks_status_cancelled</code>	Cancelling a running job flips its status and is reflected on the next poll
<code>test_cancel_job_rejects_already_finished_job</code>	Cancelling a <code>complete</code> job → 400
<code>test_cancel_job_requires_auth</code>	No auth header → 401
<code>test_get_job_returns_created_at</code>	Job responses include <code>created_at</code>

4. Notable design decisions

- **Mocking the LangGraph pipeline, not the app.** `test_e2e_job_pipeline.py` monkeypatches `api.build_pipeline` to return a fake compiled-graph stand-in whose `astream()` yields the same incremental-state-dict shape the real graph emits (`current_stage` progressing through the same values `STEP_INDEX` expects). Every other layer — job bookkeeping, `_summarize`, letterhead application, DB persistence, document/analytics endpoints — runs unmocked against the real pipeline-completion code path in `api._run_job`.
- **Async background jobs are exercised for real, not stubbed.** `POST /api/jobs` calls `asyncio.create_task(_run_job(...))`; the E2E test relies on `TestClient`'s persistent background event-loop thread (the same one production `uvicorn / asyncio.create_task` semantics assume) rather than injecting into `api.JOBS` directly, so the polling/ `wait` endpoints are tested against genuine concurrent execution.
- **Global mutable state is snapshotted and restored where any test touches it** (e.g. `config.DEVELOPER_RATES` in the rate-card tests — see [SECURITY_TESTING.pdf §4.4](#)) so no test's side effects leak into another's expectations.

5. QA Postman collection (manual/exploratory testing)

`postman/Pixous_QA_Collection.postman_collection.json` and its companion `postman/Pixous_QA.postman_environment.json` are a separate artifact from the automated pytest suite above — a real Postman v2.1 collection (not the OpenAPI export in `pixous_api_postman.json`) for manual/exploratory QA against a running server, using only synthetic/fictional data (no real client names, financial details, or personal information anywhere in it).

Setup: import both files into Postman, select the environment, fill in `admin_password` (and `qa_api_key` if you've set `QA_TEST_API_KEY`), then run **Auth** → **Login (Admin)** first — it captures the JWT into `jwt_token`, which every other request (except the QA API Key folder) sends automatically via Bearer auth. **Estimations (Manual)** → **Create Manual Estimation** and **Jobs (AI Pipeline)** → **Create Job (Text)** each capture an ID (`base_name` / `job_id`) that later requests in their folder depend on.

Folders: Auth · QA API Key (alternate auth) · Estimations (Manual) · Invoices · Jobs (AI Pipeline) · Organization & Rate Card · Documents & Analytics.

Unlike the automated E2E test (`test_e2e_job_pipeline.py`), the **Jobs (AI Pipeline)** folder is *not* mocked — it calls whatever Mistral/Groq/Gemini credentials are configured in the server's `.env` and can take 1–3 minutes per job. Everything else runs against real endpoints with no network dependency beyond the running Pixous server itself.

6. Known gaps / not covered by any of the three test documents

- `agents/*.py` (ingestion, OCR, analysis, estimation, web search, BRD, SRS, quotation node logic) are exercised only indirectly through the mocked-pipeline E2E test — their internal LLM-prompt/parsing logic has no dedicated unit tests.
- `pdf_builder.py`'s Markdown→PDF rendering (WeasyPrint) is exercised transitively via the invoice-PDF E2E assertion (`content[:4] == b"%PDF"`) but has no dedicated unit tests for its Markdown-to-CSS mapping.
- Load/performance testing (the app itself already ships a `/wait` endpoint aimed at JMeter-style blocking polls) is out of scope for all three documents — this suite is functional/security correctness only.
- See `SECURITY_TESTING.pdf` §6 for security-specific known gaps (no account-disable flag, no server-side token revocation, per-process rate limiting).